



TEL AVIV אוניברסיטת
UNIVERSITY תל אביב

Tel-Aviv University

The Lester and Sally Entin Faculty of Humanities

Linguistics Department

Evaluating Reasoning Capabilities of Large Language Models

Course Paper

Course: Language, Computation and Cognition

Instructors: Prof. Roni Katzir, Prof. Ezer Rasin

Ido Springer

207801978

29 September, 2026

1 Abstract

In recent years, Large Language Models (LLMs) shift from next token predictors to models that use intermediate output tokens to mimic human reasoning. However, despite the success of those "reasoning models" in solving complex tasks, it is still debatable whether these models develop novel reasoning capabilities during training, or they simply reflect known reasoning traces from the training data. In this short work, I review the recent advances in the LLM literature that lead to the current frontier reasoning models. Later, I review the generalization capabilities of these models, and show that their claimed reasoning capabilities are a mirage that does not generalize out-of-distribution where the training and evaluation sets are carefully controlled. In a limited experimental part, I also demonstrate the incapability of frontier models in geometric reasoning, conducting an experiment inspired by Plato's *Meno*.

2 Introduction

The current capabilities of large language models (LLMs) seem to be undeniable. Modern LLM serve as everyday code assistants to solve complex implementation tasks. In the past year, LLMs seem to have reached mathematical breakthroughs such as disproving Erdős unit distance problem (OpenAI, 2026a), progress in Navier–Stokes smoothness problem (OpenAI, 2026b), as well as solving open problems in combinatorics, algebra and computer science (OpenAI, 2026c).

These results are an outcome of a major development in the LLM area of study. Specifically, the shift from LLMs as next token predictors of natural language to models that use long series of intermediate output tokens to mimic human reasoning abilities, a method which is now coined as *test-time computing*. A particular outcome is that formal fields such as mathematics and coding, where the model outputs can be judged using a clear verifiable reward, experienced major breakthroughs.

Nevertheless, the research community is highly divided on whether these "reasoning models"¹ have actual emergent reasoning capabilities, or these models simply sample known reasoning paths that were already presented in the training data. This debate is not only technical, but touches a crucial point of the centuries-long philosophical debate of empiricism vs. rationalism; Can one teach a true reasoner using only labeled data, or that the ability to reason requires some innate mechanism?

The study of reasoning is often traced back to Plato's *Meno*. In this dialogue, Socrates leads one of Meno's uneducated slave boys to double the area of the square using deductive reasoning steps, pointing out the paradox of learnability. Plato solves this paradox by assuming recollection of innate knowledge (*anamnesis*). Chomsky famously compared the ideas described in *Meno* to one's tacit

¹I use the term "reasoning" for both logical deductive reasoning and for intermediate output token traces of LLMs, following community terms; Of course, one does not imply the other, as I show.

knowledge of language, reading *Meno* as an argument from the poverty of the stimulus, in favor of innate knowledge implemented as the Universal Grammar (Chomsky, 1988). The rise of current LLMs that are claimed to be able to reason could serve as an empiricist reading of Plato’s *Meno*: A base LLM could be compared to the slave boy, deducing its reasoning steps using intermediate chain-of-thought tokens to derive new knowledge.

In this limited review, I explore some recent works to test the claim of emergent reasoning capabilities in LLMs. I show that the answer is negative: every time the vague term ”reasoning” is restricted to a specific task of which we can control the training and evaluation data, reasoning models do not generalize out of training distribution in the way we expect from a true reasoner. For instance, they fail to apply deductive reasoning to similar problems with higher complexity, or to small changes in the input tokens.

A special attention in the LLM reasoning literature was given to methods involving Reinforcement Learning (RL) over the base model, allegedly creating some spontaneous ”aha moments” in the learning process (Guo et al., 2025). I review works that show that RL does not increase reasoning capabilities over the base model, nor composing existing capabilities the base model has.

I demonstrate these ideas in the experimental part, where I show that a reasoning model cannot correctly derive the reasoning traces of problems in geometry. Inspired by Plato’s *Meno* and Sun et al. (2026a), I show that a reasoning model that was fine-tuned on compass and ruler geometric constructions provides false proofs for constructions of some of the ancient Greek problems, which are known to be mathematically impossible to construct. The model does not seem to distinguish between possible and impossible constructions.

This short work is constructed as follows: In section 3, I elaborate about the recent evolution of LLMs from next token predictors to reasoning models. In section 4, I review works testing the generalization capabilities of frontier reasoning models, showing that the claims for reasoning are simply reflections of the training data. Section 5 is the experimental part, where I demonstrate the incapability of frontier models with geometric reasoning.

3 Constructing Reasoning Models

In this section, I describe the current training process that is used to train state-of-the-art language reasoning models. I start by describing the training steps and observed phenomena of LLMs, and later review methods to improve reasoning capabilities over the base model.

3.1 Language Modeling

Let Σ be a finite alphabet of possible tokens. A sentence will be any finite sequence $s \in \Sigma^*$ (technically with special START and END tokens). We assume a probability distribution over valid sentences $p^*(s) = p^*(w_1 \dots w_k)$. Language models aim to predict that distribution. Since $p^*(w_1 \dots w_k) = \prod_{i=1}^k p^*(w_i | w_1 \dots w_{i-1})$,

language models are defined auto-regressively, as parametric functions with parameters θ that induce a probability distribution p_θ over the next token given a sequence of previous tokens.

Given a finite empirical corpus $D \subseteq \Sigma^*$ sampled from p^* , we can optimize the parameters θ . We note that for every sequence $s = w_1 \dots w_T \in D$ we can use each subsequent token as a prediction target given the previous tokens. Therefore, we can minimize the negative log-likelihood

$$L(s; \theta) = - \sum_{t=1}^T \log p_\theta(w_t | w_{<t})$$

And the objective is averaged over all sequences $L(\theta) = \frac{1}{|D|} \sum_{s \in D} L(s; \theta)$.

Usually, the parametric function p_θ is a modern neural network architecture with a final softmax layer, and the optimization procedure to minimize the objective is obtained using back-propagation with some form of gradient-descent.

3.2 From Next Token Predictors to Reasoning Models

3.2.1 In Context Learning

Given our definition of language modeling, we would expect from a language model trained over a corpus of English sentences to complete simple English sentences in a reasonable way. For example, after the prefix *the cat sat on a ...* a possible high-probability token will be *mat* and a low-probability token will be *France* or *success*.

We don't expect however that this model could do anything else other than completing simple sentences. It has no ability to calculate arithmetic, translate sentences or answer multi-choice questions. For the model to do so, additional training is required.

Nevertheless, it was first reported by Brown et al. (2020) that GPT-3 can apply *in-context learning*. This means, the model can answer correctly tasks where the task is described only in the prompt (i.e. model input), without any additional parameter updates. For example in (1), the model (GPT-3 in this case) generalizes the translation task from the task description and the first example format to the next example.

- (1) **Input:** Translate English to French: see otter $\rightarrow$ loutre de mer, cheese $\rightarrow$
Model: fromage

In-context learning can be evaluated in zero-shot setting where only the task description is presented in the prompt, or in few-shot setting where some labeled examples are provided in the prompt preceding the test sample.

Is in-context learning a true emergent capability that arises given enough model parameters, or that the impressive results of GPT-3 are just look-up of similar tasks in the training corpora? It is hard to properly judge Brown et al. (2020) due to possible data contamination of the training data to contain possible evaluation tests. I will return to this point later when discussing generalization in reasoning models.

3.2.2 Instruction Fine-Tuning

Wei et al. (2022b) suggest additional training phase to improve the in-context learning capabilities of LLMs, nowadays called *supervised fine tuning* (SFT). In addition to the language modeling step, also called *pre-training*, where the model is exposed to the original training corpus D , Wei et al. (2022b) create a new corpus of instruction following, which I notate by D_{instruct} . This corpus contains input-output pairs (x_i, y_i) of task description and a proper answer, such as (2). The training procedure over D_{instruct} is the same as the language modeling phase, using cross-entropy objective and next-token prediction on the y_i part.

- (2) **Input:** Translate from English to Spanish: The new office building was built in less than three months.
Output: El nuevo edificio de oficinas se construyó en tres meses.

Instructions in D_{instruct} include translation, multi-choice questions, natural language inference, and other classical Natural Language Processing (NLP) tasks. The different instruction tasks in Wei et al. (2022b) are separated to training and evaluation tasks. The authors group different instruction datasets into clusters based on their task types, and use leave-one-out evaluation to measure the in-context learning results on unseen task types. The evaluations show major improvement in zero-shot in-context learning over the basic use of GPT-3 without instruction fine-tuning. This serves as evidence for cross-task generalization when training LLMs to follow basic instructions.

3.2.3 Chain of Thoughts

An important step from basic language models to reasoning models is using Chain-of-Thoughts (CoT) traces. Wei et al. (2022a) reported an interesting prompting phenomenon of in-context learning: when a base model (again GPT-3 here) is asked to answer arithmetic question, it usually fails even with few-shot in-context learning, as shown in (3). However, when the steps of the arithmetic calculation are included in the few-shot examples, in-context learning works better, as in (4).

- (3) **Input:** Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?
A: The answer is 11.
Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?
Model: A: The answer is 27.
- (4) **Input:** Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?
A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.
Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model: A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9.

The results of Wei et al. (2022a) are consistent with various types of tasks that require reasoning, such as arithmetic, commonsense, and symbolic reasoning benchmarks. In addition, Kojima et al. (2022) showed that CoT reasoning can be triggered even in zero-shot setting.

Why chain-of-thought prompting works? As Zhao et al. (2026) shows, chain of thought reasoning reflects inductive bias toward reasoning paths that were already seen in training data. Prompting with chain-of-thought obligates the model to sample from this reasoning distribution. Indeed, once the chain of thought prompting is pushed beyond the training distribution, generalization substantially drops (Zhao et al., 2026).

One implication of this view is that reasoning in LLMs can be seen as a search problem. Improving reasoning capabilities means looking for the faithful reasoning trajectories within the training data distribution. The next subsection elaborates on various methods to enhance CoT reasoning.

Note that similarly to any instruction following task, one can collect a corpus D_{CoT} composed of triplets (x_i, r_i, y_i) of a task, a chain-of-thought reasoning trace, and a final answer. The base model can be trained over D_{CoT} with next-token prediction and cross-entropy over (r_i, y_i) . This shifts the model output distribution to provide reasoning traces, thus being a foundational step in training current reasoning models.

In another view, CoT reasoning traces can be seen as resources of compute. To solve harder problems, the model would have to use more reasoning tokens at inference time. Therefore, improving the test time efficiency and scaling, thus producing more reasoning tokens, helps moving the computations from training time to inference time; Instead of improving the model by updating gradients, the computation is done by using complex reasoning traces. However, this test-time compute approach has problems. Since the reasoning traces don't have to be logical and faithful, longer reasoning traces does not necessarily mean stronger computation (Shojaee et al., 2025).

3.3 Improving Chain-of-Thought Reasoning

The fact that a model can predict tokens in the style of chain-of-thought does not indicate much about its internal reasoning capabilities. Reasoning CoT traces are token sequences and are no different from other sequences. Thus, nothing guarantees that these CoT traces will be true and logical.

However, since CoT mimic human reasoning, and using intermediate tokens to describe the path to a final answer empirically improves results, CoT can be used to improve the reasoning abilities of LLMs.

Many works from recent years try to improve model reasoning, in two major branches. One focuses on in-context learning and test-time compute scaling, without updating the model parameters. Test-time compute methods can be

classified to verifier-free and verifier based methods. The other branch focuses on post-training methods, training LLMs to provide faithful CoT traces.

3.3.1 Verifier-Free Methods

Verifier free methods improve the model reasoning based on internal consistency, without using an external verifier scoring. In the view of test-time compute, verifier free methods aim to maximize the model computation and scaling given limited token capacity. Different methods can be categorized to parallel sampling, sequential revision and internal scaling.

Parallel sampling methods are methods in which the model generates multiple solution attempts, using a rule or a score to decide the best of them. Such methods include majority voting, best-of-N sampling, beam search, or re-ranking using another LLM evaluator (Arora et al., 2026).

In sequential revision methods, the model deepens a single line of thought. The model uses additional computational cost to verify the reasoning steps. Such methods include self-correction using another forward pass, or in-context backtracking where the model is prompted to catch mistakes and fix the reasoning path (Arora et al., 2026). However, naive sequential revision might fail due to hallucinations that judge right reasoning paths as false and vice versa (Huang et al., 2024).

Internal scaling methods move the computation to the model architecture rather than using a lot of inference tokens. For example, re-using transformers layers recursively or adjusting the decoding mechanism to favor longer and descriptive sequences (Arora et al., 2026).

3.3.2 Verifier-Based Methods

In verifier based methods, the next reasoning path is selected using an external verifier and objective. Such verifiers split to Process Reward Models (PRM) and Outcome Reward Models (ORM), depending on whether the verifier judges intermediate reasoning steps or only the final answer.

The use of verifiers enhances the existing verifier-free methods. For instance, one can use an outcome reward verifier to evaluate each final output in best-of-N parallel sampling, or use a process reward verifier to enhance beam-search of reasoning steps.

A more traditional use of verifiers is using them as judges for tree and graph search methods. For examples, tree-of-thoughts (Yao et al., 2023) branches different reasoning steps in a tree, and the verifier chooses the next branch to expand using BFS/DFS. Similar methods include Monte-Carlo tree search or evolutionary search for exploring the best reasoning traces (Weng, 2025; Arora et al., 2026).

3.3.3 Post-Training and Reinforcement Learning

A different line of work does not aim to improve the model reasoning with test-time scaling. Rather, the model is post-trained to produce better reasoning

traces.

The first to improve reasoning by post-training were Zelikman et al. (2022), who used iterated fine-tuning on CoT traces that lead to the right final answer. Their procedure can be described as follows: Start with a pre-trained model M_0 and a dataset D_0 of few-shot prompts. In each training iteration k , for each question-output pair $(x_i, y_i) \in D$, the model M_k generates CoT trace $\hat{r}_i$ and a final answer $\hat{y}_i$ based on x_i . If the prediction is true and $\hat{y}_i = y_i$, the triplet $(x_i, \hat{r}_i, y_i)$ is added to D_k . Otherwise the model is wrong, and in that case we let the model M_k try again to generate a CoT trace and answer $(\hat{r}_i', \hat{y}_i')$, this time using the true answer y_i . If the model succeeds and $\hat{y}_i' = y_i$, the triplet $(x_i, \hat{r}_i', y_i)$ is added to D_k . Lastly, we create M_{k+1} by training M_0 on D_k . That way, in the fine-tuning process the model receives signals from both right and wrong CoT traces. Weng (2025) shows that the training program of Zelikman et al. (2022) is an approximation to the policy-gradient algorithm in reinforcement learning.

Special attention is given to Reinforcement Learning (RL) for improving reasoning, following the success of Deepseek-R1 model (Guo et al., 2025). Using RL is similar to verifier-based methods of test-time compute, however in this case the model parameters are iteratively updated. RL was previously used as a method for preference optimization of language models from human feedback (Ouyang et al., 2022) to align the models to be helpful and not-harmful. In the case of reasoning, for some formal tasks like math and coding, a special reward model is not necessary, and the model can be improved using rules alone. Deepseek-R1 is trained using RL with Verifiable Rewards (RLVR), with two rule-based rewards: a format reward for wrapping the reasoning with `< think > ... < /think >` tokens, and accuracy reward for whether the final answer is correct.

DeepSeek claimed that with pure RL over the base model and no instruction-tuning stage, it is possible to learn advanced reasoning capabilities, with radical claims for emergent "aha moments" in solving equations (Guo et al., 2025). Note that the use of RL and accuracy rewards alone is indeed a major shift from post-training LLMs with cross-entropy loss, which has been common to all methods cited here so far. The RLVR model can generate any sequence of reasoning tokens without teacher forcing. Does the use of RL endow the model with external reasoning abilities that were not seen during training? I discuss this question in the next section.

4 Generalization in Reasoning Models

So far, I described the basic building blocks of current reasoning models. But I ignored a crucial question: do these models actually reason, in the way we expect from a logical reasoner? In this section I show that while reasoning models have impressive performance on complicated tasks, when it comes to out of distribution generalization, the answer is false. I focus on several cases of generalization to unseen examples: sensitivity to input variations, and gen-

eralization to harder tasks than seen in training. In addition, I discuss the role of RL in reasoning.

4.1 Prompt Sensitivity

Logical reasoning is an abstract formal process. Therefore, a reasoner should not be affected by small input variations. However, LLMs reasoning seems to be probabilistic pattern matching rather than true formal reasoning. One evidence for this is the sensitivity of reasoning models to small variations in the input tokens.

Mirzadeh et al. (2025) demonstrate this fragility in a series of variations over the Grade School Math (GSM8k) dataset. They extend this dataset, which contains simple math question with chain-of-thought solutions, to a dataset of symbolic templates where the names and the numbers in the questions can be controlled. They show that for the same math question, the performance of LLM have unwarranted variance when changing names and numbers, questioning the reliability of GSM8k as a math reasoning benchmark, and the reasoning process of the model. In addition, they show that by inserting additional but irrelevant information to the question, state-of-the-art model performance dramatically drops. This reveals that LLMs lack the ability to distinguish between relevant and irrelevant information for problem solving.

While Mirzadeh et al. (2025) control the variations of the evaluation data, Zhao et al. (2026) control also the training data. They provide an isolated and controlled environment to train LLMs from scratch, which allows them to systematically probe the models under various distribution conditions. Given that, CoT reasoning is tested using three different distribution dimensions: task, length and format. Zhao et al. (2026) show that CoT reasoning vanishes when it is pushed beyond training distributions, supporting the hypothesis that CoT reasoning in LLMs is a mirage that simply reflects reasoning paths that were seen during training.

4.2 Increasing Complexity

Similarly to input variations, a formal reasoning process should not be affected by the complexity of the problem. Given a certain problem in the training data, true generalization requires the ability to solve a more complicated form of the same problem in test time. However, LLMs fail to supply this reasoning generalization criterion.

Shojaee et al. (2025) construct controllable puzzle environments that allow increasing the problem complexity while maintaining the core logical problem constraints, such as Towers of Hanoi, Checker Jumping and River Crossing. They show that state-of-the-art reasoning models face a complete accuracy collapse beyond certain complexities across different environments. By investigating the reasoning trace itself, they show that LLMs fail to perform exact computation, meaning LLMs provide inconsistent reasoning when using explicit algorithms across scales and problems. Moreover, they show that beyond a

complexity point, reasoning models fixate on early errors and cannot recover. This provides strong evidence against the test-time compute approach: longer reasoning traces do not imply better computation. On the other hand, critics claim that the work of Shojaei et al. (2025) evaluates only state-tracking and not reasoning, and limits in state-tracking performance as complexity increases are acceptable (Lawsen, 2025). This is related to the competence-performance distinction. LLMs lack competence (Katzir, 2023), therefore they only mimic the execution steps of exact algorithms instead of learning the algorithm itself using some internal mechanism.

4.3 Levels of Generalization

The recent success of RLVR reasoning models such as DeepSeek-R1 (Guo et al., 2025) in professional level mathematics benchmarks calls for careful evaluation of these models. The evaluation tests in the previous works I mentioned so far in this section demonstrate the general concept of lack of true generalization out-of-distribution. However, the strict requirement for controllable datasets reduces the evaluations to simple toy tasks.

Sun et al. (2026a) keep the same requirement of controlling the increasing complexity and diversity in evaluation, but do use challenging math problems instead of toy tasks. Their suggested benchmark explores three axis of out of distribution generalization. For each axis they construct train-test pairs of problems to isolate a specific reasoning capability:

- **Exploratory:** assessing whether models can apply known problem-solving skills to more complex instances of the same problem domain.
- **Compositional:** evaluating models ability to combine distinct reasoning skills, previously learned in isolation, to solve novel problems that require integration of these skills.
- **Transformative:** testing whether models can adopt unconventional strategies by moving beyond familiar approaches to solve problems.

Examples for math problems of each category are detailed in Table 1. For example, exploratory generalization requires inferring from octagon to dodecagon. Compositional generalization requires combining the concepts of greatest common divisor (GCD) and polynomial factorization. The most interesting example is the transformational generalization, that requires finding roots of quintic polynomials. A basic result in Galois Theory shows this is not always possible to solve with elementary mathematical operations, and one have to find the real roots with complex methods. In the experimental section I suggest another task based on this idea.

The empirical results of Sun et al. (2026a) show that as the mathematical task complexity increases, frontier models performance collapse to near-zero despite substantial test-time compute. Investigation of the CoT traces show, similarly to Shojaei et al. (2025), that reasoning models are lost in self-verification

and fixating on wrong solutions, so their test-time computation is not efficient. Moreover, Sun et al. (2026a) show that reasoning models trained with RL fail to generalize to high complexity problems. Most importantly, RL reasoning models do not seem to compose skills or to have transformational "aha moments" that would allow them to solve compositional and transformational generalization problems, unlike human mathematicians.

Generalization Type	Task	Training Problem	Test Problem
Exploratory	GEOMETRIC REASONING	Find the number of rectangles that can be formed inside a fixed regular <i>octagon</i> where each side of the rectangle lies on either a side or a diagonal of the <i>octagon</i> .	Find the number of rectangles that can be formed inside a fixed regular <i>dodecagon</i> where each side of the rectangle lies on either a side or a diagonal of the <i>dodecagon</i> .
Compositional	POLYNOMIAL GCD	(A) What is the highest common factor of 21385 and 2961? (B) $f(x) = -36x^3 + 272x^2 - 412x - 240$. Find real roots of $f(x)$.	Find the greatest common divisor of the two polynomials $f(x) = x^5 + x^3 - x^2 - 1$ and $g(x) = x^4 - 3x^2 - 4$.
Transformative	POLYNOMIAL ROOTS	Solve $f(x) = 36x^3 + 272x^2 - 412x - 240$.	Solve $f(x) = x^5 + 10x^3 + 20x - 4$.

Table 1: Examples of train and test problems for every generalization type, from Sun et al. (2026a).

4.4 The Role of Reinforcement Learning

Even though I reviewed some works that show the lack of generalization in reasoning models once the reasoning is formally tested in controlled settings, one could argue that the use of reinforcement learning in current methods, and

verifiable rewards (RLVR) in particular, can endow the model with emergent reasoning capabilities that were not seen during training. First, note that as I mentioned, the use of RL to post-train LLMs is indeed a new mechanism which is inherently different from supervised teacher forcing with cross-entropy loss. Guo et al. (2025) famously claimed that DeepSeek-R1-zero, that was trained with RL alone over the base model (without SFT), shows spontaneous "aha moment", leading to a new method for solving an equation. However, as in previous cases, recent evidence shows that RL does not create new reasoning methods that were not seen in the training data. It's again a mirage caused by the shift in the output distribution, sampling some reasoning traces more often.

Yue et al. (2025) show that changing the evaluation metric can shed light on the role of RL in post-training. Usually, reasoning models are evaluated with accuracy metric (pass@1), which averages the number of tasks the model succeeds to solve in a given dataset. However, this metric risks underestimating the true potential of a model, especially if it fails on hard problems, despite being capable of solving them with more sampling. Yue et al. (2025) suggest the pass@k metric, allowing multiple time (k) sampling for each problem, and a problem is considered solved if any of the k sampled outputs is correct. This gives a more robust view of the model reasoning capabilities.

Using the pass@k metric, Yue et al. (2025) show with extensive experiments some surprising findings. It seems that RLVR training narrows the reasoning coverage instead of expanding it. Although RLVR models outperform their base models at small k , the base models consistently surpass RLVR models across all benchmarks and LLM families as k increases. For most problems, the base model can produce at least one correct CoT trace for large k , but the RLVR phase suppresses these solutions. Moreover, investigating the CoT traces of RLVR models indicates that reasoning paths generated by an RLVR model already exist in the distribution of its base model. Hence, RLVR does not introduce any novel reasoning capabilities.

Nevertheless, some still claim in favor of RL for enhancing reasoning. Sun et al. (2026b) claim for "RL grokking" phenomenon, where RL post-trained model can sometimes solve a problem of which the base model has pass@k=0 accuracy in a controlled environment. Yet, this phenomenon seems to be subtle, and requires using extended RL technics such as replay, warm-up and continuous rewards. This might need further examination.

5 Experimental Work

In Plato's *Meno*, Socrates leads one of Meno's slave boys in the process of solving a geometric riddle: to double the area of a given square. Inspired by this dialogue, and by the work of Sun et al. (2026a), I wanted to check if I can train LLMs some geometric reasoning, and in particular ruler and compass constructions.

Ruler and compass constructions, similarly to finding the roots of quintic polynomial in Sun et al. (2026a), can contain some transformational obstacle:

not all length and angles can be built by ruler and compass, as Galois Theory shows. In particular, some construction problems that the ancient Greek could not solve, proved centuries later to be impossible. For example, constructing a regular heptagon, squaring the circle, or trisecting an angle.

In this toy experiment, I wanted to examine the reasoning traces of LLMs in transformational impossible problems. In particular: does a model that was trained to construct a regular triangle, square, pentagon and hexagon, will understand that the construction of a heptagon is impossible? Will it provide a false proof, or will it acknowledge the impossibility of this construction? A false proof will serve as evidence to the hypothesis that the model only sample reasoning traces from the training data, instead of applying properly self-verification reasoning.

Note that this is not a proper comparison to the transformational problems of Sun et al. (2026a), since all of their problems are indeed solvable, however with novel methods that were not seen in training data. Here, all problems in the training data are solvable, and the test data is not always solvable.

5.1 Experimental Details

The experiment was very limited and designed to be restricted due to lack of computing resources. The code for the experiment was written using AI assistance (GLM-5.3-Flash) and was validated by myself.

5.1.1 Dataset

For the ruler and compass construction data, I used the recently published Geometric Generative Reasoning Benchmark (GGBench, Wei et al. (2026)). This dataset contains 1,411 geometric construction problems, alongside with CoT reasoning trace for each problem and final GeoGebra code for constructing the example. The dataset was split to 1308 training samples and 100 heldout samples. Three samples were omitted due to very long reasoning trace of the provided solution. I validated that the training data contains the constructions of every regular polygon up to six sides, and does not contain constructions of more complex regular polygons.

5.1.2 Model and Training

For the reasoning model, I used a version of Qwen-7B that was distilled from DeepSeek-R1, thus containing 7B parameters. Instead of training all model parameters, I used Quantized Low Rank Adaptation (QLoRA, Dettmers et al. (2023)) in 4-bit quantization with default Unsloth hyper-parameters. The adapter was trained on T4 GPU with 16GB memory on Google Colab platform, using the entire training data, for one epoch.

5.2 Selected Constructions

For qualitative CoT analysis, I tested the trained model with four constructions. The exact input prompts are in the appendix.

1. **Building a regular heptagon.** This construction is not possible, as it requires building the angle $\frac{2\pi}{7}$ which requires extracting a cubic root which is not possible.
2. **Building a regular octagon.** This construction is possible, as a power of 2. Octagons were not seen in training, calling for exploratory generalization.
3. **Angle Trisection.** This construction is not possible in the general case since it requires a cubic root.
4. **Double the size of a given square.** This construction is possible and compares the trained model with uneducated slave boy.

5.3 Qualitative Results

The trained model provides constructions for all four problems. All four construction proofs are completely false, revealing substantial hallucinations in geometric formal reasoning. The shortened version of the model output appears in the appendix. In addition, the model provides GeoGebra code which does not compile for any problem.

When building a regular heptagon, the model does not build the required angle $\frac{2\pi}{7}$. Instead, it uses the angle 60° as a false approximation, and by the end of the proof, hallucinates the heptagon vertices. When building a regular octagon, the model tries to build an angle of 45° that should be replicated to eight vertices, however, the angle construction is hallucinated. For the task of angle trisection, the model first tries to trisect a line, but then falsely claims that this helps to trisect an angle. When doubling a square, the model at first do focus on the square diagonal, but fails to extend this diagonal into the target square.

The model does not seem to distinguish the possible and the impossible proofs. When the model is stuck reaching a mid-destination of the proof, it simply assumes the desired target, without providing any explanation, or sometimes with false explanation.

To conclude, qualitative examination of the CoT traces indicate no evidence to true geometric understanding, but to a collection of known geometric facts that do not form together into logical and coherent proofs. This supports the view that LLMs simply mimic reasoning paths that were seen in training data. Note however the limitations of this experiment, and in particular possible lack of training iterations.

6 Conclusion

While frontier reasoning LLMs seem to solve very challenging problems, careful examination of their training process, and controlling precisely their training and evaluation data, reveals that when it comes to out of distribution generalization, "reasoning" of LLMs is a mirage caused by upper-sampling of existing reasoning traces from the training set. Moreover, special methods originally aimed at enhancing reasoning capabilities, such as RLVR, in practice tend to narrow down the space of possible reasoning traces instead of expanding it. Generalization in reasoning problems could be classified to exploratory, compositional and transformative generalization. Exploratory generalization is limited as problem complexity increases, and frontier reasoning models do not seem to overcome the compositional nor the transformational jumps as humans do. This is reflected in the literature I reviewed, as well as in the geometric reasoning experiment.

References

- Arora, Akhil et al. (2026). "Current Advances in LLM Reasoning". In: *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 5: Tutorial Abstracts)*.
- Brown, Tom et al. (2020). "Language models are few-shot learners". In: *Advances in Neural Information Processing Systems* 33, pp. 1877–1901.
- Chomsky, Noam (1988). *Language and Problems of Knowledge: The Managua Lectures*. Current Studies in Linguistics Series 16. Cambridge, MA: MIT Press. ISBN: 9780262031332.
- Dettmers, Tim et al. (2023). "QLoRA: Efficient Finetuning of Quantized LLMs". In: *Thirty-seventh Conference on Neural Information Processing Systems*.
- Guo, Daya et al. (2025). "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning". In: *arXiv preprint arXiv:2501.12948*.
- Huang, Jie et al. (2024). "Large Language Models Cannot Self-Correct Reasoning Yet". In: *The Twelfth International Conference on Learning Representations*.
- Katzir, Roni (2023). "Why large language models are poor theories of human linguistic cognition: A reply to Piantadosi". In: *Biolinguistics* 17, pp. 1–12.
- Kojima, Takeshi et al. (2022). "Large Language Models are Zero-Shot Reasoners". In: *Advances in Neural Information Processing Systems*. Vol. 35, pp. 22199–22213.
- Lawson, A. (2025). "The Illusion of the Illusion of Thinking: A Comment on Shojaee et al. (2025)". In: *arXiv*. DOI: 10.48550/arxiv.2506.09250.
- Mirzadeh, Iman et al. (2025). "GSM-Symbolic: Understanding the Limitations of Mathematical Reasoning in Large Language Models". In: *International Conference on Learning Representations*. Ed. by Y. Yue et al. Vol. 2025, pp. 94743–94765.

- OpenAI (2026a). *An OpenAI model has disproved a central conjecture in discrete geometry*. URL: <https://openai.com/index/model-disproves-discrete-geometry-conjecture/>.
- (2026b). *On the Navier–Stokes Millennium Prize Problem*. URL: <https://openai.com/index/navier-stokes-solution/>.
- (2026c). *Ten advances in mathematics and theoretical computer science*. URL: <https://openai.com/index/ten-advances-in-mathematics/>.
- Ouyang, Long et al. (2022). “Training language models to follow instructions with human feedback”. In: *Advances in Neural Information Processing Systems* 35, pp. 27730–27744.
- Shojaee, Parshin et al. (2025). “The Illusion of Thinking: Understanding the Strengths and Limitations of Reasoning Models via the Lens of Problem Complexity”. In: *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- Sun, Yiyu et al. (2026a). “OMEGA: Can LLMs Reason Outside the Box in Math? Evaluating Exploratory, Compositional, and Transformative Generalization”. In: *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.
- Sun, Yiyu et al. (2026b). “RL Grokking Recipe: How Does RL Unlock and Transfer New Algorithms in LLMs?” In: *The Fourteenth International Conference on Learning Representations*.
- Wei, Jason et al. (2022a). “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models”. In: *Advances in Neural Information Processing Systems*. Vol. 35, pp. 24824–24837. DOI: 10.52202/068431–1800.
- Wei, Jason et al. (2022b). “Finetuned Language Models are Zero-Shot Learners”. In: *International Conference on Learning Representations*.
- Wei, Jingxuan et al. (2026). “GGBench: A Geometric Generative Reasoning Benchmark for Unified Multimodal Models”. In: *Conference on Computer Vision and Pattern Recognition 2026*.
- Weng, Lilian (May 2025). “Why We Think”. In: *lilianweng.github.io*. URL: <https://lilianweng.github.io/posts/2025-05-01-thinking/>.
- Yao, Shunyu et al. (2023). “Tree of Thoughts: Deliberate Problem Solving with Large Language Models”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 36.
- Yue, Yang et al. (2025). “Does Reinforcement Learning Really Incentivize Reasoning Capacity in LLMs Beyond the Base Model?” In: *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- Zelikman, Eric et al. (2022). “STaR: Bootstrapping Reasoning With Reasoning”. In: *Advances in Neural Information Processing Systems*. Vol. 35, pp. 15476–15488.
- Zhao, Chengshuai et al. (2026). “Is chain-of-thought reasoning of llms a mirage? a data distribution lens”. In: *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 15231–15261.

7 Appendix

Here are the prompts that were used for the selected geometric constructions, and a shortened version of the model output, after I removed the generated GeoGebra code. All proofs are completely false.

Q1: Given a circle with center O and a point A on its circumference, construct a regular heptagon ABCDEFG inscribed in the circle.

Step 1: Construct a line through center O perpendicular to radius OA. *Method:* 1. Draw line segment OA. 2. Construct a line passing through O perpendicular to OA. *Principle:* The construction requires a specific angle to be inscribed. The angle between adjacent vertices is $2\pi/7$ radians. To find point B such that $\angle AOB = 2\pi/7$, we first construct an angle of 60° at the center to find a starting point.

Step 2: Construct an angle of 60° at the center O. *Method:* 1. Select center O and point A. 2. Construct a 60° angle with vertex O, one side being OA. *Principle:* This is a standard construction to create a 60° angle using a compass and straightedge. The ray intersects the circle at point B.

Step 3: Find vertex C by rotating point A by 60° . *Method:* 1. Rotate A around O by 60° to find point C. 2. Segment OC equals OA. *Principle:* Since the heptagon is regular, vertices are equally spaced. The central angle is $2\pi/7$. Since 60° is not exactly $2\pi/7$ ($\approx 102.86^\circ$), this is a starting approximation.

Step 4: Construct the chord AC to find chord length. *Method:* 1. Draw segment connecting A and C. 2. Segment AC represents a chord. *Principle:* The length of chord AC is determined by the central angle $\angle AOC$ (60°). This chord serves as a reference to determine the heptagon's side length.

Step 5: Find the midpoint of the chord AC. *Method:* 1. Construct the midpoint of chord AC (point M). *Principle:* The perpendicular bisector of a chord passes through the circle's center. Midpoint M is the foot of the perpendicular from center O to chord AC.

Step 6: Construct a new circle to locate the next vertex. *Method:* 1. Draw a circle with center M and radius MA. 2. This new circle intersects the original circle at point D. *Principle:* The distance from O to M is the distance from O to chord AC. The new circle's radius is the chord's half-length (MA). The intersection defines point D such that $\angle AOD = 2\pi/7$. This is the first step toward the heptagon.

Step 7: Construct the regular heptagon. *Method:* 1. Connect points A, C, D in a cyclic manner. 2. The resulting polygon ABCDEFG is a regular heptagon inscribed in the circle. *Principle:* By constructing D such that $\angle AOD = 2\pi/7$, we created a central angle corresponding to the side length of a regular heptagon. Connecting vertices forms the required polygon.

Q2: Given a circle with center O and a point A on its circumference, construct a regular octagon ABCDEFGH inscribed in the circle.

Step 1: Construct the first side of the octagon. *Method:* 1. Draw a circle centered at the given center O that passes through the given vertex A. 2. Find the intersection points of this new circle with the original circle. Let's label one of these points B. 3. The segment AB is the first side of the regular octagon. *Principle:* The side length of a regular polygon inscribed in a circle is equal to the chord length subtended by a central angle of 360° divided by the number of sides. For an octagon, this is $360^\circ/8 = 45^\circ$. The chord length can be found by intersecting two circles with radii equal to the circle's radius and a central angle of 45° .

Step 2: Find the next vertex of the octagon. *Method:* 1. Construct an angle of 45° with its vertex at the center O and one side along the radius OA. 2. The intersection of this angle's second side with the circle gives the next vertex, C. *Principle:* This step uses a rotation transformation to locate the next vertex at a 45° angle from the previous one. The angle is determined by the formula $\theta = 360^\circ/n$, where n is the number of sides.

Step 3: Construct the remaining vertices. *Method:* 1. Repeat the angle construction process from the previous vertex (B). Draw a 45° angle with vertex at O and one side along the segment OB. 2. The intersection of this new angle's second side with the circle gives the next vertex, D. 3. Continue this process for each subsequent vertex (E, F, G, H) by drawing a 45° angle with the last constructed side as its radius. *Principle:* Each vertex is constructed by rotating the previous vertex by 45° around the center O. This ensures that each adjacent vertex is separated by a central angle of 45° , which is the required angle for a regular octagon.

Step 4: Complete the octagon and add final annotations. *Method:* 1. Connect the vertices

A, B, C, D, E, F, G, and H in sequence to form the regular octagon. 2. Add angle markers to show the central angles (each 45°) and the inscribed angles (each 22.5°). 3. Add a right-angle marker at the center O to emphasize that the central angle is 45° , and the inscribed angle is half of that. *Principle:* Connecting the constructed vertices forms the required polygon. Annotations visually confirm that the construction has met the problem's requirements, verifying the correctness of the method.

Q3: Given an angle $\angle BAC$, construct two rays that divide the angle into three equal parts.

Step 1: Draw an auxiliary circle centered at vertex A. *Method:* 1. Choose a point B on one of the rays of the angle. 2. Draw a circle with center A and radius AB. *Principle:* This circle will be used to transfer the length of the radius to the other rays, creating equal segments.

Step 2: Divide the radius AB into three equal parts. *Method:* 1. Define a point P on the ray AB such that the distance from A to P is equal to the radius AB. 2. Use the Segment tool to draw a segment from A to P. This segment AP is equal in length to AB. 3. Construct the midpoint of the segment AP and label it Q. 4. Construct the midpoint of the segment PQ and label it R. *Principle:* By construction, the segments AQ, QR, and RB are all equal in length, each being $1/3$ of the original radius AB.

Step 3: Draw the first trisection ray. *Method:* 1. Draw a ray from vertex A through the first division point R. 2. This ray, ray AR, divides the angle $\angle BAC$ into two equal angles: $\angle BAR$ and $\angle RAC$. *Principle:* By the construction in Step 2, the segment AR is exactly one-third of the radius AB. The central angle corresponding to a $1/3$ segment of a circle is $1/3$ of the full angle, which is 120° . Therefore, $\angle BAR = 120^\circ$.

Step 4: Construct the second trisection ray. *Method:* 1. Construct a circle centered at A with radius AR (the first trisection segment). 2. Find the intersection points of this new circle with the original circle centered at A with radius AB. Let one intersection be S. 3. Draw a ray from A through S. This ray, ray AS, is the second trisection ray. *Principle:* The angle subtended by an arc at the center of a circle is proportional to its length. Since the arc from R to S is equal to the arc from A to R, the central angle $\angle RAS$ is also 120° . The ray AS divides the original angle $\angle BAC$ into three equal parts: $\angle BAR$, $\angle RAS$, and $\angle SAC$.

Step 5: Finalize the construction and add annotations. *Method:* 1. Draw the two trisection rays, ray AR and ray AS. 2. Construct the three resulting angles: $\angle BAR$, $\angle RAS$, and $\angle SAC$. 3. Set the colors of the new angles to match the original angle for clarity. 4. Add angle markers to visually confirm that all three angles are equal. *Principle:* This step completes the construction by visually representing the trisection of the original angle. The annotations verify that the construction successfully divides $\angle BAC$ into three equal parts.

Q4: Given a square ABCD, construct another square that has twice the area of ABCD.

Step 1: Construct the diagonal of the original square. *Method:* 1. Draw a line segment connecting vertex A and vertex C of the original square ABCD. This segment is the diagonal of the square. *Principle:* The diagonal of a square is a line segment that connects two non-adjacent vertices. It serves as the key to constructing a square with double the area.

Step 2: Find the midpoint of the diagonal. *Method:* 1. Construct the midpoint of the diagonal segment AC. *Principle:* The midpoint of the diagonal of a square is also the center of the square. This point will serve as the center for the new, larger square.

Step 3: Construct a circle with diameter AC. *Method:* 1. Draw a circle with the center at the midpoint M and a radius equal to the distance from M to any of the original vertices (e.g., A). *Principle:* According to Thales's theorem, any angle inscribed in a semicircle is a right angle. The circle with diameter AC ensures that any point on its circumference forms a right angle with the endpoints of the diameter.

Step 4: Determine the side length of the new square. *Method:* 1. Find the intersection points of the circle with the line containing the side AB of the original square. 2. Let E be one of these intersection points. 3. The length of segment AE is the side length of the new square. *Principle:* The new square must have its sides at a 45-degree angle to the sides of the original square. The intersection of the circle and the side AB provides the vertex E of the new square.

Step 5: Construct the new square and complete the figure. *Method:* 1. Draw a circle with center A and radius AE. 2. Find the intersection points of this new circle with the line containing side AD. Let F be one of these points. 3. Connect points A, E, F, and the corresponding point on the other side to form the vertices of the new square. 4. Draw the diagonals of the new square to match the style of the original figure. *Principle:* By construction, the side length of the new square (AE) is equal to the length of the diagonal of the original square (AC). Since the

area of a square is proportional to the square of its side length, the new square has an area of $2 \times (AC)^2/2 = 2 \times (\text{side length of ABCD})^2$, which is twice the area of the original square.
